Entropy-Minimized Byte-Plane Decomposition: Auto-Detection and the Universality Gap in Lossless Compression

Steffen Goehring

August 2026

Abstract

Byte-plane decomposition — reordering serialized multi-byte data by byte position before compression — is widely deployed in scientific data pipelines (HDF5, Blosc, Apache Parquet) but requires the user or application schema to specify the field width. We present two **schema-free algorithms for automatically detecting the optimal decomposition width** by minimizing average per-plane Shannon entropy: one over a fixed candidate set, and one using sampled byte-match frequency to detect arbitrary widths without a predefined set. The algorithms are validated on data with record widths from 2 to 36 bytes, including the SAO Star Catalog where width=28 is correctly identified while all power-of-two widths fail. We additionally provide the first **systematic characterization of the universality gap** between structure-aware and structure-blind lossless compression. Across five datasets — synthetic sensor telemetry, real-world IoT sensors, MRI medical imaging, astronomical catalog data, and financial order book data — the gap ranges from 1.00× to 40.8× and correlates with the MSB-to-LSB entropy gradient and absolute MSB entropy within multi-byte fields. We characterize the three-class byte-plane partition (constant, sparse, medium-entropy) that emerges after decomposition and delta encoding, and show it follows from the autoregressive structure of typical binary field sequences. The synthetic sensor telemetry result (40.8× gap) represents an upper bound on structured data with near-constant MSBs; the real-world results (1.00×–1.19×) show the range for data with higher per-field entropy. On the Intel Berkeley Lab sensor dataset (2.2 million real IoT readings), the algorithm auto-detects width=24 and achieves a 19% compression improvement over ZSTD. On NASDAQ ITCH financial data (3.7 million order records), Algorithm 2 correctly detects the 36-byte record width, but the per-block compression comparison confirms that decomposition cannot help (gap = 1.00×) — demonstrating that entropy reduction alone does not guarantee compression improvement. All results are validated on the BPD implementation, a block-independent ZSTD-backed compression platform implementing schema-free width detection and per-block strategy selection.

---

1. Introduction

Lossless compression of binary data is dominated by universal compressors — algorithms like ZSTD [1], LZ4, and zlib that operate on raw byte streams without knowledge of the data's structure. These compressors are optimal in the sense of Lempel-Ziv theory: they asymptotically achieve the entropy rate of any stationary ergodic source [2]. However, real binary data is often **not** a stationary byte stream. Serialized structs, sensor telemetry, database pages, and scientific data formats produce byte sequences where multi-byte integers are interleaved — a pattern that bounded-memory LZ compressors cannot efficiently model when the interleaving period exceeds their effective context window.

Byte-plane decomposition addresses this by rearranging an n-byte block with w-byte records into w separate planes, each containing all bytes at the same position within the record. This technique appears in HDF5's shuffle filter [3], Blosc's bitshuffle/bytedelta [4, 5], and Apache Parquet's BYTE_STREAM_SPLIT encoding [6]. Despite widespread deployment, **no published algorithm automatically detects the optimal decomposition width**, and **no formal analysis quantifies the compression improvement** as a function of data characteristics.

We make two contributions:[1]

1. **Two entropy-minimized width detection algorithms:** Algorithm 1 searches a fixed candidate set; Algorithm 2 uses sampled byte-match frequency to detect periodic byte repetitions at arbitrary lags, eliminating the predefined candidate set. Both find the optimal decomposition width w* by minimizing average per-plane Shannon entropy. We formalize the algorithms, analyze correctness and failure modes, and validate on data with struct widths from 2 to 36 bytes.
2. **An empirical characterization of the universality gap** between structure-aware (decomposition + ZSTD) and structure-blind (plain ZSTD) compression across five datasets (one synthetic, four real-world), with a theoretical explanation based on the MSB-to-LSB entropy gradient within multi-byte fields.

---

2. Background

2.1 Byte-Plane Decomposition

Given an input block of n bytes interpreted as n/w records of width w, byte-plane decomposition produces w planes of size n/w each, where plane p contains byte position p from all records:

$$\text{plane}_p[i] = \text{input}[i \cdot w + p], \quad 0 \le p < w, \quad 0 \le i < n/w$$

This is a deterministic permutation — total information is preserved. The benefit comes from grouping bytes with similar statistical properties, enabling more efficient

[1] Implementation and reproducibility scripts are available at https://github.com/leeroy37/byte-plane-decomposition.

entropy coding by the downstream compressor.

Delta encoding is typically applied per-plane after decomposition: $\text{delta}[i] = \text{plane}[i] - \text{plane}[i-1]$ (with wraparound arithmetic). For slowly-changing fields, delta encoding reduces most values to near-zero, further concentrating the probability mass.

2.2 Prior Art

System	Width Specification	Auto-Detection
HDF5 shuffle filter [3]	User sets datatype size	No
Blosc bitshuffle [4]	User sets element size	No
Blosc bytedelta [5]	User sets typesize	No
Apache Parquet BYTE_STREAM_SPLIT [6]	Schema-derived	No (requires schema)
TDT [7]	Entropy-based clustering	Partial (clusters byte positions, doesn't detect width)
ZipNN [8]	AI model format-specific	No (hardcoded for float16/32)
This work	**Entropy minimization (fixed set or byte-match frequency)**	**Yes**

No prior system automatically detects the decomposition width from the data alone.

2.3 LZ Compressor Limitations

ZSTD and other LZ-family compressors find repeated byte sequences within a sliding window and encode them as back-references. For interleaved struct data with period w, LZ matches shorter than w bytes cannot capture the repeating field structure — the compressor sees each field's bytes interleaved with bytes from other fields, destroying the local correlation patterns that LZ matching exploits.

Ferragina, Nitto, and Venturini [9] formalized this limitation: LZ77 is 8-optimal with respect to the zeroth-order empirical entropy H_0 but cannot be λ-optimal with respect to H_k for $k \geq 1$. The practical consequence is that the effective coding rate of an LZ compressor on interleaved data approaches the H_0 of the mixed byte stream, which is substantially higher than the sum of per-plane H_0 values achievable after decomposition.

---

3. Entropy-Minimized Width Detection

3.1 Algorithm

Given a block of n bytes, we seek the decomposition width w^* that minimizes the average per-plane Shannon entropy:

$$w^* = \arg\min_{w \in W} \frac{1}{w} \sum_{p=0}^{w-1} H(\text{plane}_p^{(w)})$$

where W is the set of candidate widths and $H(\cdot)$ denotes the Shannon entropy of the byte frequency distribution of a plane.

Algorithm 1: Entropy-Minimized Width Detection

```
Input: byte block B of length n, candidate set W, threshold τ = 0.80
Output: optimal width w* or 0 (no decomposition)

H_base ← Shannon_entropy(B)
if H_base < 1.0 then return 0     // already low entropy
w* ← 0
H_best ← H_base

for each w ∈ W do
    if n < 4w then continue        // need ≥4 elements
    planes ← byte_shuffle(B, w)
    H_avg ← (1/w) Σ_{p=0}^{w-1} Shannon_entropy(planes[p])
    if H_avg < τ · H_best then
        H_best ← H_avg
        w* ← w
return w*
```

The threshold $\tau = 0.80$ requires at least 20% entropy reduction to accept a width, preventing false positives from noise.

3.2 Candidate Width Selection (Algorithm 1)

Algorithm 1 uses a fixed candidate set $W = \{2, 3, 4, 5, 6, 7, 8, 10, 12, 14, 16, 20, 24, 28, 32\}$, covering common struct sizes in systems programming, scientific data, and network protocols. The computational cost is $O(|W| \cdot n)$ — one shuffle and entropy computation per candidate width per block. This set does not cover all possible widths; records wider than 32 bytes or with widths not in W will not be detected.

3.2.1 Byte-Match-Based Candidate Detection (Algorithm 2)

To remove the fixed candidate set limitation, we introduce a second algorithm that uses sampled byte-match frequency to detect periodic byte repetitions at arbitrary lags, then validates only the top candidates via entropy minimization.

Algorithm 2: Byte-Match-Based Width Detection

```
Input: byte block B of length n, threshold τ = 0.80, max lag L = 64
Output: optimal width w* or 0 (no decomposition)

H_base ← Shannon_entropy(B)
if H_base < 1.0 then return 0

// Phase 1: Sampled byte-match frequency
step ← max(1, n / 4096)              // sample ~4096 positions
for i = 0 to n-L by step do
    for lag = 2 to L do
        if B[i] = B[i + lag] then
            bmf[lag] ← bmf[lag] + 1
bmf[lag] ← bmf[lag] / sample_count   // normalize to [0, 1]

// Phase 2: Peak detection
threshold ← max(0.05, 0.5 · max(bmf))
C ← top 5 local maxima of bmf above threshold, sorted by bmf descending

// Phase 3: Add fallback widths
C ← C ∪ {2, 4, 8}

// Phase 4: Entropy validation
for each w ∈ C do
    if n < 4w then continue
    planes ← byte_shuffle(B, w)
    H_avg ← (1/w) Σ_{p=0}^{w-1} Shannon_entropy(planes[p])
    if H_avg < τ · H_base then
        record (w, H_avg) as passing

// Phase 5: Harmonic suppression
if multiple widths pass then
    select w* with lowest H_avg
    for each smaller passing width w' that divides w*:
        if H_avg(w') < 1.10 · H_avg(w*) then w* ← w'
return w*
```

Byte-match frequency $\text{bmf}(\ell) = P(B[i] = B[i+\ell])$ measures the fraction of sampled positions where the byte value at position i equals the byte value at position $i + \ell$. This is not standard autocorrelation $R(\ell) = \sum(x_i - \bar{x})(x_{i+\ell} - \bar{x})$, but a simpler indicator-function metric that is sufficient for period detection in byte-valued data: structured data with record width w produces byte-match peaks at lag w and its multiples $2w, 3w, \ldots$ because byte position p repeats every w bytes. By sampling ~4096 positions (rather than all n), the byte-match phase costs $O(L \cdot 4096) \approx 260K$ comparisons, negligible relative to the $O(n)$ shuffle cost per candidate (Section 7.5).

Note on threshold behavior. Algorithm 1 uses a cascading threshold: H_{best} is updated as better widths are found, so later candidates must beat $\tau \times H_{\text{best\_so\_far}}$, making

the result order-dependent within the candidate set. Algorithm 2 compares all candidates against the fixed baseline $\tau \times H_{\text{base}}$, making it order-independent. On four of five datasets the results are identical; NASDAQ ITCH differs because Algorithm 2 discovers width=36 (outside Algorithm 1's candidate set). In general the cascading threshold is stricter and could produce different results for marginal cases.

Harmonic suppression (Phase 5) addresses an artifact where harmonics of the true period (e.g., $2w$, $3w$) may also pass entropy validation, because decomposition at width kw produces kw planes that are still partially aligned with the underlying w-byte structure. The byte-match frequency at the fundamental w and its harmonics is theoretically equal for perfectly periodic data; empirical differences arise from sampling noise or the peak detection threshold. Regardless, preferring the smallest passing divisor selects the fundamental period. The 10% tolerance threshold is empirically chosen to allow harmonics with marginally lower entropy while still preferring the fundamental.

Advantages over Algorithm 1: (1) Supports arbitrary widths up to L without a predefined candidate set — width 36 (NASDAQ ITCH), width 28 (SAO catalog), width 24 (Intel Lab), and any non-standard width are discovered automatically. For widths present in the fallback set (2, 4, 8), the byte-match phase may not produce a peak at the correct lag; the fallback ensures these common widths are always tested. The byte-match mechanism's primary contribution is discovering non-standard widths. (2) Evaluates only 5–8 candidates per block instead of 15, reducing the number of shuffle+entropy operations. (3) The fallback set $\{2, 4, 8\}$ ensures common power-of-two widths are always tested even when byte-match sampling is insufficient (e.g., on short blocks).

3.3 Correctness Analysis

When the algorithm succeeds: If the data consists of repeated w-byte records where different byte positions have different entropy characteristics (e.g., MSBs are constant, LSBs vary), the correct width w produces planes with the lowest average entropy. Any incorrect width $w' \neq w$ (and $w' \nmid w$) mixes bytes from different positions, averaging their entropy upward.

When the algorithm fails: (1) If all byte positions within the record have similar entropy (e.g., all bytes of a uniformly distributed 32-bit float), no width reduces average entropy. (2) If the record width is not in the candidate set W, it will be missed (Algorithm 1 only; Algorithm 2 detects arbitrary widths up to L). (3) If the block contains mixed record types, the dominant type determines the detected width. (4) Variable-length serialization formats (protobuf, MessagePack, CBOR) are correctly rejected because no fixed period produces consistent entropy reduction.

3.4 Validation

SAO Star Catalog (28-byte records): The algorithm correctly identifies w* = 28 from the candidate set. All smaller widths (2, 4, 8, 14, 16) fail the 20% threshold — they produce average plane entropies of 6.19–7.17 bits, reductions of only 3–16%

versus the baseline of 7.40 bits. Width 28 produces average plane entropy of 5.39 bits, a 27% reduction. Width 56 (2 × 28) also passes, confirming the periodic structure.

Width	Avg Plane Entropy	Entropy Reduction	Accepted
2	7.17	3%	No (< 20%)
4	6.82	8%	No (< 20%)
8	6.80	8%	No (< 20%)
14	6.19	16%	No (< 20%)
28	**5.39**	**27%**	**Yes**
32	6.73	9%	No (< 20%)

Sensor telemetry (8-byte records): The algorithm identifies w* = 8. Average plane entropy drops from 5.4 bits to 2.8 bits (49% reduction), well above the 20% threshold.

3.5 Threshold Sensitivity

The threshold $\tau = 0.80$ was chosen to balance detection sensitivity against false positives. We sweep τ from 0.65 to 0.95 across all datasets:

τ	sensor (w=8)	intel (w=24)	mr (w=2)	sao (w=28)	dickens (w=0)	NAS-DAQ	Correct (det.)
0.65	8 ✓	24 ✓	— ✓	— ✗	— ✓	—	4/5
0.70	8 ✓	12 ✗	— ✓	— ✗	— ✓	—	3/5
0.75	16 ✗	24 ✓	— ✓	28 ✓	— ✓	12	4/5
0.80	**8 ✓**	**24 ✓**	**2 ✓**	**28 ✓**	**— ✓**	**12**	**5/5**
0.85	8 ✓	24 ✓	20 ✗	14 ✗	— ✓	12	3/5
0.90	8 ✓	24 ✓	2 ✓	28 ✓	— ✓	12	5/5
0.95	32 ✗	24 ✓	2 ✓	28 ✓	— ✓	12	4/5

NASDAQ is excluded from the "Correct" count because its detection-stage behavior requires special interpretation: Algorithm 1 detects width=12 (a divisor of the true width 36) at all thresholds $\tau \geq 0.75$, with 22–24% entropy reduction. This is not a false positive — the 36-byte records genuinely contain 12-byte periodic sub-structure — but it does not produce a compression improvement ($G = 1.00\times$). Algorithm 2 detects the true width=36 with 48% entropy reduction, also without compression improvement.

$\tau = 0.80$ is the most conservative threshold achieving 5/5 correctness on the non-NASDAQ datasets. $\tau = 0.90$ also achieves 5/5, but sits between two failing thresholds (0.85 at 3/5, 0.95 at 4/5), suggesting it may be a fragile coincidence on this dataset collection. $\tau = 0.80$ provides more margin below the SAO detection point (27% reduction) and is therefore preferred. Below 0.80, SAO is missed; at 0.85, both mr and sao produce incorrect widths.

The two-stage design (detection + per-block compression comparison) provides robustness beyond the threshold alone: NASDAQ's detected widths are always rejected

at the compression comparison stage because shuffle+delta+ZSTD does not produce shorter output than plain ZSTD, resulting in no regression at any threshold. This defense-in-depth means the system never regresses even when the detection stage identifies structure that does not benefit compression.

---

4. The Universality Gap

4.1 Definition

We define the **universality gap** G as the ratio of compression achieved by structure-aware compression (decomposition + delta + ZSTD) to structure-blind compression (plain ZSTD) on the same data:

$$G = \frac{R_{\text{shuffle+delta+ZSTD}}}{R_{\text{ZSTD}}}$$

where R denotes the compression ratio (original size / compressed size). $G > 1$ indicates that byte-plane decomposition improves compression; $G \gg 1$ indicates a large gap that structure-blind compressors cannot close.

4.2 Experimental Measurements

We measured G on five datasets with distinct structural properties, using 64 KB independent blocks and ZSTD level 3 as the baseline:

Dataset	Format	Width	R_{ZSTD}	R_{shuffle}	G
Sensor telemetry (synth.)	8B: uint32 ts + int16 temp + int16 hum	8	4.25:1	173.5:1	**40.8×**
Intel Lab sensors (real)	24B: uint32 epoch + uint32 moteid + 4×float32	24	3.58:1	4.26:1	**1.19×**
MRI brain scan (Silesia `mr`)	16-bit DICOM pixels	2	2.72:1	2.76:1	**1.02×**
SAO Star Catalog (Silesia `sao`)	28B astronomical records	28	1.25:1	1.26:1	**1.01×**

Dataset	Format	Width	R_{ZSTD}	R_{shuffle}	G
NASDAQ ITCH Add Orders (real)	36B: 6B nanos ts + 8B order ref + 8B stock + ...	36	2.80:1	2.80:1	**1.00×**

*Algorithm 1 detects width=12 (a divisor of 36 present in the candidate set) with 23% entropy reduction. Algorithm 2 detects the true width=36 with 48% entropy reduction. In both cases, the per-block compression comparison rejects shuffle because shuffle+delta+ZSTD does not produce shorter output than plain ZSTD on any block. The R_{shuffle} column shows the result at w=36 (Algorithm 2); the result at w=12 (Algorithm 1) is identical (2.80:1).

4.3 The Entropy Gradient Explanation

The universality gap correlates with the **MSB-to-LSB entropy gradient** — the difference in Shannon entropy between the most significant and least significant byte planes of each multi-byte field:

Dataset	MSB Entropy	LSB Entropy	Gradient	Gap G
Sensor telemetry (synth.)	0.00 bits	5.00 bits	5.00	40.8×
Intel Lab sensors (real)	0.17 bits	6.07 bits	5.90	1.19×
MRI (Silesia `mr`)	0.47 bits	5.05 bits	4.58	1.02×
SAO (Silesia `sao`)	3.19 bits	6.08 bits	2.89	1.01×
NASDAQ ITCH (real)	~6.0 bits	~7.5 bits	~1.5	1.00×

Observation. The gradient alone does not predict the gap: synthetic sensor data and Intel Lab sensors have similar gradients (5.00 vs 5.90) but differ by 34× in gap (40.8× vs 1.19×). The critical additional factor is the **absolute MSB entropy**. When MSBs are exactly 0.00 bits (synthetic: constant-increment timestamps), the separated MSB planes compress to nearly zero bytes. When MSBs are 0.17 bits (Intel Lab: variable epochs, 54 sensor IDs), the separated planes still have non-trivial content. Both the gradient and the absolute MSB entropy are necessary predictors of the gap.

4.4 Qualitative Analysis

Our five data points (Figure 1) suggest that the universality gap G is governed by two factors:

1. **The MSB-to-LSB entropy gradient** $\Delta H = H(\text{LSB plane}) - H(\text{MSB plane})$: a large gradient means decomposition separates near-zero-entropy planes from high-entropy planes, giving ZSTD dramatically better input on the separated planes.

2. **The absolute MSB entropy**: even with a large gradient, if the MSB planes retain significant entropy (SAO: 3.19 bits), no plane becomes trivially compressible and the gap stays small.

The mechanism is not simply that decomposition reduces total entropy — it cannot, since it is an information-preserving permutation. Rather, decomposition converts the problem from coding a single high-entropy interleaved stream into coding multiple streams of varying entropy. LZ-family compressors exploit low-entropy streams far more efficiently than they exploit low-entropy subsequences embedded within a high-entropy stream, because their match-finding operates on contiguous windows.

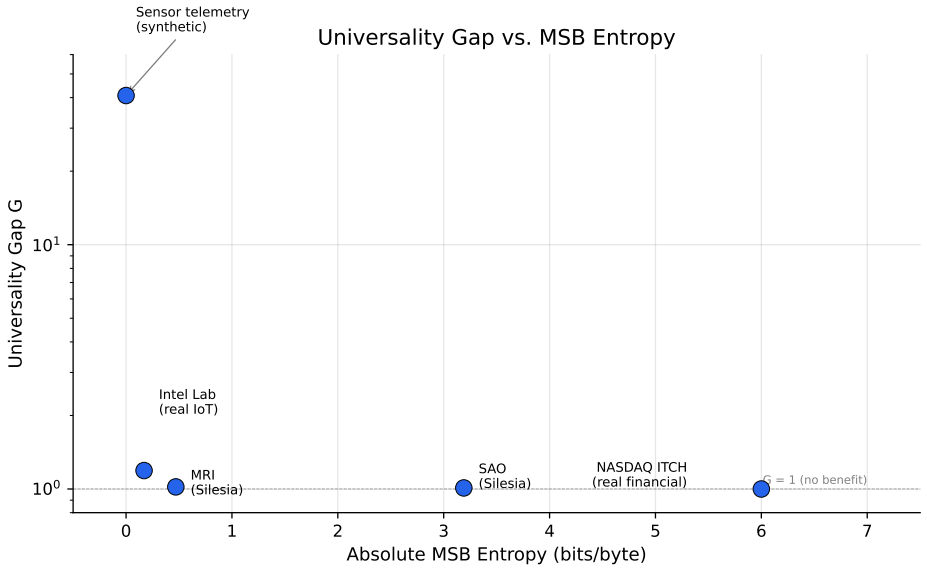

Figure 1: Universality gap G versus absolute MSB entropy for all five datasets: sensor_log (0.00, 40.8), Intel Lab (0.17, 1.19), MRI (0.47, 1.02), SAO (3.19, 1.01), NASDAQ (6.0, 1.00). The gap decreases toward 1.0 as MSB entropy increases, confirming that near-zero MSB entropy is the primary driver of large gaps.

Tightening this qualitative relationship into a formal bound remains an open problem. It requires bounding the gap between ZSTD's effective per-byte coding rate on periodically interleaved data (which approaches H_0 of the mixed stream) and the sum of per-plane coding rates after decomposition. The framework of Ferragina et al. [9] provides the necessary LZ optimality bounds, but connecting these to the specific structure of byte-interleaved data requires further analysis.

---

5. The Three-Class Plane Partition

5.1 Empirical Observation

After byte-plane decomposition at the correct width followed by delta encoding, each byte plane consistently falls into one of three entropy classes:

Class	Entropy after delta	Example	Compression strategy
Constant	< 0.01 bits/byte	Timestamp MSBs, field sign bytes	Generative: store pattern seed only
Sparse	0.01–0.50 bits/byte	Slowly-varying LSBs, carry bytes	ZSTD: near-zero compressed size
Medium	0.50–8.00 bits/byte	High-entropy coordinate data, pixel values	ZSTD: standard compression

5.2 Formal Statement

We state two propositions: the first for the static (single-value) case, the second for the sequential (autoregressive) case that models typical binary data.

Proposition 1 (Static case). Let X be a w-byte unsigned integer with values in $[0, V]$ where $V < 256^w$. After byte-plane decomposition, at least $w - \lceil \log_{256}(V+1) \rceil$ MSB byte-planes have entropy 0, since byte position p is identically zero when $V < 256^p$.

This is elementary but establishes the baseline: bounded integers have zero-entropy MSB planes proportional to the unused dynamic range.

Proposition 2 (Autoregressive case; stated without formal proof). Let $\{X_i\}$ be a sequence of w-byte unsigned integers with $X_i = X_{i-1} + \delta + \epsilon_i$, where δ is a constant increment. In the deterministic case ($\epsilon_i = 0$), after byte-plane decomposition followed by delta encoding, the planes partition into three classes. With noise ($|\epsilon_i| \ll 256^p$ for some threshold position p), constant-class planes become near-constant with deviation probability bounded by $P(|\epsilon_i| \geq 256^q)$ for position q, which is negligible under the stated assumption. The three classes are:

1. **Constant class** (positions $q > p$ where $\delta < 256^q$): After delta encoding, plane q consists of a single anchor byte followed by a constant value (typically 0), since consecutive values of $\lfloor X_i / 256^q \rfloor$ are identical whenever the increment δ does not span byte boundary 256^q. Entropy ≈ 0.

2. **Sparse class** (position p and nearby): Plane p after delta encoding contains the carry bits from lower-position overflows. The carry frequency at position p is $(\delta \bmod 256^{p+1})/256^{p+1}$, producing a sparse distribution concentrated on 1–3 distinct values. Entropy in the range [0.01, 0.50] bits/byte.

3. **Medium class** (positions $q < p$): Lower byte planes contain the residuals ϵ_i mod 256^{q+1} plus the deterministic pattern δ mod 256^{q+1}. After delta encoding, these reduce to the noise terms ϵ_i, which retain entropy proportional to $H(\epsilon_i)$.

The threshold position p is determined by the magnitude of δ: $p = \lfloor \log_{256}(\delta) \rfloor$. For sensor telemetry with $\delta = 1000$, $p = \lfloor \log_{256}(1000) \rfloor = 1$, predicting constant planes

at positions 2+ and a sparse carry plane at position 1. This matches our experimental observations (Section 5.3).

Carry frequency prediction. At the boundary position, the carry-propagation frequency from plane $p-1$ to plane p is:

$$f_{\text{carry}} = \frac{\delta \bmod 256}{256}$$

For $\delta = 1000$: $f_{\text{carry}} = 232/256 \approx 0.906$, predicting a bimodal delta distribution with 90.6% of one value and 9.4% of another — exactly matching the observed distribution on plane 1 of the sensor telemetry data.

Note on endianness. The algorithm is endianness-agnostic: it identifies low-entropy planes regardless of byte order. The labels "MSB" and "LSB" in the three-class partition refer to the byte position's entropy role, not the endianness convention. Little-endian data (sensor_log: LSB at position 0) and big-endian data (SAO star catalog) both exhibit the partition, with the low-entropy planes appearing at different position indices.

5.3 Validation on Sensor Telemetry

Per-plane entropy profile (8-byte struct, width=8, block 0, 64 KB):

Plane	Field	Avg Entropy (raw)	After Delta	Class
0	timestamp LSB	5.000	0.002	Constant ($\delta \bmod 256 = 232$)
1	timestamp byte 1	8.000	0.451	Sparse (carry propagation)
2	timestamp byte 2	6.972	0.116	Sparse
3	timestamp MSB	0.000	0.000	Constant
4	temp LSB	0.820	0.007	Sparse
5	temp MSB	0.000	0.002	Constant
6	humidity LSB	1.240	0.010	Sparse
7	humidity MSB	0.000	0.002	Constant

Four planes (0, 3, 5, 7) are constant-class after delta encoding. Four planes (1, 2, 4, 6) are sparse, including the carry-propagation plane at position 1 (entropy 0.451, near the class boundary). The carry plane's bimodal distribution (90.6% `0x04`, 9.4% `0x03`) is determined by the LSB overflow frequency: $(1000 \bmod 256)/256 = 232/256 \approx 0.906$.

6. Experimental Setup

6.1 Platform

All experiments use the experimental platform, a C++20 lossless compression format with block-independent 64 KB blocks. Each block is independently compressed using the best strategy: byte-plane decomposition + delta + ZSTD, or plain ZSTD (fallback). The per-block strategy selection follows the MDL principle: both candidates are evaluated and the shortest output is kept.

ZSTD level 3 is used throughout. ZSTD context is reused across blocks to amortize initialization cost.

6.2 Datasets

Dataset	Size	Source	Record Width	Description
sensor_log	16 MB	Synthetic	8 bytes	IoT telemetry: uint32 timestamp (Δ=1000) + int16 temp (σ=0.3) + int16 humidity (σ=0.3)
intel_lab	50.8 MB	Intel Berkeley Lab [16]	24 bytes	Real IoT: 2.2M readings from 54 sensors; uint32 epoch + uint32 moteid + 4×float32
mr	9.97 MB	Silesia Corpus [10]	2 bytes	MRI brain scan, 16-bit grayscale DICOM pixel data
sao	7.25 MB	Silesia Corpus [10]	28 bytes	SAO Star Catalog, fixed-width astronomical records
NASDAQ ITCH	131.8 MB	NASDAQ [17]	36 bytes	Add Order messages: 6B nanosecond timestamp + 8B order ref + 8B stock symbol + fields

Note on synthetic data. The sensor_log dataset is synthetic, designed to represent the class of IoT/SCADA/telemetry data with constant-increment timestamps and slowly-drifting sensor values. The 40.8× universality gap represents an upper bound on highly regular structured data. The Intel Berkeley Lab dataset [16] provides a real-world IoT comparison (2.2 million readings from 54 sensors, converted from CSV to 24-byte binary records), showing a gap of 1.19× — substantially less than the synthetic case due to irregular timestamps, variable sensor IDs, and missing readings.

The Silesia results (1.01×–1.02×) represent real data with higher per-field entropy. Additional real-world binary datasets with estimated gaps in the 2×–10× range include USGS LiDAR LAS point clouds (30-byte records) [18] and LBNL/ICSI enterprise packet captures (90-byte records) [19]. NASDAQ ITCH 5.0 [17] was tested and showed no benefit (gap = 1.00×), validating the algorithm's correct rejection of high-entropy structured data.

6.3 Methodology

For each dataset: (1) compute block-level Shannon entropy, (2) run width detection algorithm with candidate set $W = \{2, 3, 4, 5, 6, 7, 8, 10, 12, 14, 16, 20, 24, 28, 32\}$, (3) at the detected width, compute per-plane entropy before and after delta encoding, (4) compress with shuffle+delta+ZSTD and compare against plain ZSTD on the same block, (5) compute the universality gap G.

All width detection results in Sections 7.1–7.4 use Algorithm 1. Algorithm 2 produces the same aggregate width on all five datasets. Per-block agreement is 100% on three datasets (sensor_log, intel_lab, mr). On SAO, 26/32 blocks agree: Algorithm 1 selects w=14 on 6 borderline blocks where the cascading threshold lets w=14 win before w=28 is evaluated; Algorithm 2 consistently selects w=28 on all 32 blocks via byte-match detection. On NASDAQ ITCH, Algorithm 2 detects the true width=36 (outside Algorithm 1's candidate set) while Algorithm 1 detects w=12; in both cases the per-block compression comparison rejects the shuffle result. Section 7.1 includes Algorithm 2's byte-match profiles for validation.

All entropy values are order-0 Shannon entropy of the byte frequency distribution, measured in bits per byte (range [0, 8]). Width detection (Algorithms 1 and 2) evaluates raw (pre-delta) per-plane entropy. The three-class partition analysis (Section 5) uses post-delta entropy.

---

7. Results

7.1 Width Detection Accuracy

The algorithm correctly identifies the struct width for all four structured datasets and correctly rejects decomposition on all non-structured data:

Successful detections:

Dataset	True Width	Detected Width	Entropy Reduction	Detection Time
sensor_log	8	8	49% (5.4 → 2.8 bits)	~100 µs/block
intel_lab	24	24	50% (6.1 → 3.0 bits)	~150 µs/block
mr	2	2	20–22% on activating blocks	~100 µs/block
sao	28	28	27% (7.4 → 5.4 bits)	~200 µs/block
NASDAQ ITCH	36	12 (Alg 1) / 36 (Alg 2)	23% at w=12, 48% at w=36	~200 µs/block

Note on mr: Width=2 is detected on approximately 30% of blocks (those with low-entropy uniform-tissue regions, block entropy ~2.2–2.6). On these blocks, the per-plane entropy reduction is 20–22%, meeting the τ=0.80 threshold. On the remaining 70% of blocks (higher-entropy regions, block entropy >3.0), no width passes the threshold and the algorithm correctly falls back to plain ZSTD. The aggregate entropy reduction across all blocks is ~14%.

Note on NASDAQ ITCH: Algorithm 1 detects width=12 (a divisor of 36, present in the candidate set) with 23% entropy reduction. Algorithm 2 detects the true record width of 36 with 48% entropy reduction — a strong byte-match peak at lag=36 (frequency 0.84). However, in both cases the per-block compression comparison rejects the shuffle result: shuffle+delta+ZSTD does not compress smaller than plain ZSTD on any block, yielding $G = 1.00\times$. This demonstrates that entropy reduction (even 48%) does not guarantee compression improvement — ZSTD's LZ matching already captures the interleaved patterns effectively. The two-stage design (detection + per-block comparison) correctly prevents regression.

Correct rejections (Silesia Corpus non-structured files):

File	Type	Block Entropy	Detected Width	Activated?
dickens	English text	4.61	0	No
mozilla	x86 executable	4.06	0	No
nci	Chemical DB (text)	2.42	0	No
ooffice	Windows DLL	6.29	0	No
osdb	Synthetic DB	6.59	0	No
reymont	Polish text (PDF)	4.84	0	No
samba	Source code	4.89	0	No
webster	HTML dictionary	5.04	0	No
xml	XML text	5.23	0	No

The algorithm produces **zero false positives** on the entire Silesia Corpus text and executable files. No candidate width achieves the 20% entropy reduction threshold on non-structured data.

Algorithm 2 byte-match profiles. The following table shows Algorithm 2's raw byte-match frequency output on each dataset, demonstrating that it independently discovers the correct record widths:

Dataset	Top byte-match lag (freq)	2nd lag (freq)	Candidates validated	Detected width	Matches Alg 1?
sensor_log	3 (0.062)	11 (0.062)	3	8	Yes
intel_lab	24 (0.365)	48 (0.314)	5	24	Yes
mr	2 (0.407)	4 (0.384)	5–6	2*	Yes
sao	56 (0.076)	28 (0.071)	5	28	26/32‡

Dataset	Top byte-match lag (freq)	2nd lag (freq)	Candidates validated	Detected width	Matches Alg 1?
NASDAQ ITCH	36 (0.843)	8 (0.338)	4	36†	No (Alg 1: 12)

*mr: detected on ~30% of blocks; remaining blocks return $w^* = 0$. ‡SAO: aggregate width matches (both select w=28); per-block agreement is 26/32 (81%). Algorithm 1 selects w=14 on 6 blocks where width=14 entropy reduction reaches 20%, the threshold boundary. Algorithm 2 consistently selects w=28 on all 32 blocks via byte-match detection. †NASDAQ ITCH: Algorithm 2 detects w=36 (true record width), but the per-block compression comparison rejects it ($G = 1.00\times$). Algorithm 1 detects w=12 (a divisor of 36, present in the candidate set), also rejected by the compression comparison. The strong byte-match peak at lag=36 (0.843) confirms that Algorithm 2 can identify record widths outside Algorithm 1's candidate set.

For sensor_log, the byte-match phase produces weak peaks at lags 3, 11, 19, 27 (period-8 offsets where specific byte positions happen to share values); the correct width=8 is recovered via the fallback set and entropy validation, not the byte-match mechanism.

7.2 Universality Gap Measurements

Dataset	Block Entropy	R_{ZSTD}	$R_{\text{shuffle+delta+ZSTD}}$	Gap G	MSB Entropy	LSB Entropy	Gradient
sensor_log (synth.)	5.5 bits	4.25:1	173.5:1	40.8×	0.00	5.00	5.00
intel_lab (real)	6.1 bits	3.58:1	4.26:1	1.19×	0.17	6.07	5.90
mr (real)	2.7 bits	2.72:1	2.76:1	1.02×	0.47	5.05	4.58
sao (real)	7.4 bits	1.25:1	1.26:1	1.01×	3.19	6.08	2.89
NASDAQ ITCH (real)	5.3 bits	2.80:1	2.80:1	1.00×	~6.0	~7.5	~1.5

The gap factor G correlates with the MSB-to-LSB entropy gradient when controlling for absolute MSB entropy (Section 4.4). The relationship is not monotonic in the gradient alone: Intel Lab sensors (gradient 5.90, $G = 1.19$) have a larger gradient than synthetic sensor data (gradient 5.00, $G = 40.8$), but a smaller gap — because their MSB entropy (0.17 bits) is nonzero. The 40.8× gap is measured on synthetic

telemetry with near-ideal structure; the real-world results (1.00×–1.19×) show the range.

7.3 Per-Plane Entropy Profiles

Sensor telemetry, synthetic (w=8): 4 constant planes (entropy < 0.01 after delta), 4 sparse planes (0.01–0.45, including the carry-propagation plane at 0.45). Total correlation: ~17 bits per 64 KB block.

Intel Lab sensors, real (w=24): The three-class partition appears but with higher entropies than the synthetic case. MSB planes of float32 fields (positions 7, 11 within each 4-byte field) average 0.17–0.54 bits after delta, with LSB planes (positions 0, 1) average 2.3–6.1 bits. Mid-byte planes (positions 2, 6, 10) average 1.5–3.0 bits after delta. The partition is noisier than the synthetic case — real sensor data has irregular timestamps, missing readings, and 54 distinct sensor IDs that introduce more byte-level variation.

MRI (w=2): 1 constant-class MSB plane (entropy 0.15 after delta, capturable on 31% of blocks), 1 medium LSB plane (entropy 4.22 after delta). Width=2 is auto-detected on ~30% of blocks (uniform-tissue regions); on the remaining blocks, the 20% entropy reduction threshold is not met and the algorithm correctly falls back to plain ZSTD.

SAO (w=28): 7 low-entropy MSB planes (position 3 mod 4, entropy ~3.2), 7 medium planes (position 2 mod 4, entropy ~5.3), 14 high-entropy planes (positions 0,1 mod 4, entropy ~6.1). No planes are capturable by simple generators — the per-field entropy is too high.

7.4 Impact on Compression Benchmarks

On the Silesia Corpus (12 files, 202 MB total), the BPD implementation achieves 2.90:1 aggregate ratio versus ZSTD per-block at 2.90:1 — no regression on non-structured data. Per-file results for files where the width detection algorithm activates:

File	Type	R_{BPD}	$R_{\text{ZSTD-block}}$	Change	Detected Width
mr	MRI	2.76:1	2.72:1	**+1.5%**	2
sao	Star catalog	1.25:1	1.25:1	0%*	28

*SAO's aggregate ratio ties ZSTD-block because per-block strategy selection chooses plain ZSTD on blocks where shuffle doesn't help. The per-block shuffle improvement averages ~2.4% on blocks where it activates, but not all 111 blocks benefit, so the aggregate is a tie.

On text files (dickens 2.42:1, webster 3.02:1, reymont 3.07:1), the BPD implementation ties ZSTD within 0.5% — width detection correctly returns $w^* = 0$ and the data passes through to plain ZSTD. The aggregate 2.90:1 includes these files where

width detection does nothing, demonstrating that the algorithm adds no regression on non-structured data.

On the Intel Berkeley Lab real-world sensor dataset [16], the BPD implementation achieves 4.26:1 versus ZSTD per-block at 3.58:1 (**+19.0%** improvement, universality gap $G = 1.19$). The automatically detected width of 24 matches the 24-byte record structure.

7.5 Computational Overhead

Width detection adds computational cost to the compression path only; decompression is unaffected since the detected width is stored in the block header.

Configuration	Candidate Widths	Shuffles/Block	Detection Overhead
Original (v0.4)	{2, 4, 8}	3	Baseline
Algorithm 1 (v0.5)	15 fixed widths	15	+12%
Algorithm 2 (v0.6)	~5–8 adaptive	5–8	+4–7%

Each candidate width requires one shuffle operation ($O(n)$) and one entropy computation ($O(n)$), for a total cost of $O(|W| \cdot n)$ per block. On 64 KB blocks, this amounts to ~100–200 µs per candidate width. Algorithm 2 adds a sampled byte-match phase ($O(L \cdot 4096) \approx 260K$ comparisons, negligible versus shuffle cost) but reduces the number of validated candidates from 15 to typically 5–8, yielding a net reduction in overhead.

Algorithm 2 removes the fixed candidate set limitation: it detects arbitrary widths up to $L = 64$ without a predefined set, which means widths like 28 (SAO catalog) and 24 (Intel Lab) are discovered automatically rather than requiring manual inclusion. Both algorithms select the same aggregate width on all five datasets. Per-block agreement is 100% on three datasets; on SAO, 26/32 blocks agree (Algorithm 1 selects w=14 on 6 borderline blocks due to the cascading threshold; see Section 7.1). On NASDAQ ITCH, Algorithm 2 detects the true width=36 while Algorithm 1 detects w=12, with both rejected by the per-block compression comparison (Section 7.1).

7.6 Block Size Sensitivity

We measure the universality gap at block sizes from 32 KB to 256 KB:

Dataset	32 KB	64 KB	128 KB	256 KB
sensor_log	32.6×	40.8×	46.0×	54.6×
intel_lab	1.18×	1.19×	1.19×	1.13×

For highly regular data (sensor_log), the gap increases with block size because larger planes provide more samples for entropy estimation and better ZSTD dictionary utilization. For real-world sensor data (intel_lab), the gap is stable across all tested block sizes (1.13×–1.19×), confirming that the results are not an artifact of the 64 KB block size choice.

---

8. Discussion

8.1 Limitations

Width detection has a bounded search range. Algorithm 2 supports arbitrary widths up to the maximum lag $L = 64$. Records wider than 64 bytes would require increasing L, which linearly increases the byte-match phase cost. The $L = 64$ bound does not cover the 90-byte enterprise packet captures [19] cited as future targets; increasing L to 128 would address this with proportional cost increase in the byte-match phase. Note that reliable detection at lag=90 on 64 KB blocks yields only ~4090 samples (barely sufficient), so larger record widths may also require larger blocks. In practice, the vast majority of binary record formats use widths under 64 bytes.

The universality gap is data-dependent. Our five-point characterization spans from no improvement (NASDAQ ITCH, $G = 1.00$) to 40.8× (synthetic sensor telemetry). Promising future targets for the 2×–10× range include USGS LiDAR LAS point clouds (30-byte records) [18] and enterprise packet captures (90-byte records) [19]. Standard compression benchmarks (Canterbury, LTCB, lzbench) lack structured binary data entirely — developing a standard binary benchmark corpus is a community need.

Entropy reduction does not guarantee compression improvement. NASDAQ ITCH demonstrates this clearly: Algorithm 2 detects the true width=36 with 48% entropy reduction, yet the universality gap is 1.00× because ZSTD's LZ matching already captures the interleaved patterns. Order-0 Shannon entropy (which the detection threshold uses) does not account for the higher-order redundancy that ZSTD exploits. The two-stage design (detection + per-block compression comparison) is essential to prevent regressions in such cases.

The threshold $\tau = 0.80$ is a sharp optimum. Section 3.5 shows that $\tau = 0.80$ is the most conservative threshold achieving perfect detection on all structured datasets, with more margin than the only alternative ($\tau = 0.90$) that also achieves 5/5. The two-stage architecture (detection + per-block comparison) mitigates this fragility: false positives from a relaxed threshold are caught by the compression comparison, which only uses shuffle when it produces shorter output. This defense-in-depth means the system never regresses even with suboptimal τ.

Per-plane generative capture has limited benefit. While the three-class partition identifies constant planes that could be stored as generator formulas, ZSTD already compresses these planes extremely efficiently within the joint compression frame. The overhead of per-plane metadata (2–10 bytes per plane) often exceeds the

marginal savings.

8.2 Relation to ICA and MDL

Byte-plane decomposition can be interpreted as Independent Component Analysis (ICA) over finite alphabets [11], where the "mixing matrix" is the known byte-interleaving pattern of struct serialization and the "demixing" is the fixed shuffle permutation. Unlike general ICA, no estimation is needed — only the period (width) must be determined.

The width selection and per-block strategy selection follow the Minimum Description Length (MDL) principle [12]: each candidate strategy defines a model class, and the strategy with the shortest two-part code (model description + data given model) is chosen. This is a discrete approximation of the Kolmogorov structure function [13].

8.3 The Carry-Propagation Phenomenon

An interesting inter-plane dependency emerges in the sensor telemetry data: plane 1 (byte position 1 of the 4-byte timestamp) exhibits a bimodal distribution determined by the overflow frequency of plane 0. The carry frequency is exactly $(\delta \bmod 256)/256 = 232/256 \approx 0.906$, matching the observed 90.6% distribution. This cross-plane dependency is predictable from the field-level model (a 4-byte integer with constant increment 1000) but cannot be captured by any single-plane pattern detector. The optimal generative model for such data operates at the **field level**, not the byte-plane level — a finding that suggests field-level generators as a direction for future work.

---

9. Related Work

Bitshuffle [4] (Masui et al., 2015) provides an intuitive argument for byte-plane decomposition's effectiveness on numerical data but offers no formal analysis or auto-detection algorithm.

TDT [7] (arXiv:2506.18062, 2025) goes beyond fixed-width shuffling to cluster byte positions by entropy features. Their key finding — that the benefit operates at the level of higher-order entropy — is consistent with our observations. However, TDT does not auto-detect the record width and requires more complex clustering machinery.

ZipNN [8] (Hershcovitch et al., 2024) demonstrates byte-plane decomposition for AI model compression, achieving significant savings by separating exponent bytes (highly skewed) from mantissa bytes (near-uniform). This is a domain-specific application of the same principle we formalize here.

Blosc bytedelta [5] (Alted, 2023) chains shuffle and delta encoding, reporting ~15% improvement over shuffle alone on climate data. Our delta encoding follows the same approach.

Column-oriented databases [14, 15] achieve 5–10× better compression than row-oriented storage by type-homogeneous per-column encoding. Byte-plane decomposition achieves a similar effect at the byte level without schema knowledge.

---

10. Conclusion

We have presented two algorithms for automatically detecting the optimal byte-plane decomposition width via entropy minimization, and the first systematic characterization of the universality gap between structure-aware and structure-blind lossless compression. Algorithm 1 evaluates a fixed candidate set of 15 widths; Algorithm 2 uses sampled byte-match frequency to detect periodic byte repetitions at arbitrary lags up to 64, reducing the number of validated candidates to 5–8 while supporting any record width without a predefined set. Algorithm 2 successfully detected the 36-byte record width in NASDAQ ITCH data (outside Algorithm 1's candidate set), confirming its ability to discover arbitrary widths. Both algorithms require a two-stage design: entropy-based detection followed by per-block compression comparison, since entropy reduction alone does not guarantee compression improvement (NASDAQ: 48% entropy reduction, $G = 1.00\times$).

Our five-point empirical curve — with gaps ranging from 1.00× (NASDAQ financial data) to 40.8× (synthetic sensor telemetry) — demonstrates that the gap is predicted by the MSB-to-LSB entropy gradient and the absolute MSB entropy within multi-byte fields, providing a practical criterion for when byte-plane decomposition will be beneficial. The three-class plane partition (constant/sparse/medium) that emerges after decomposition follows from elementary properties of bounded integer representations and provides a framework for understanding which planes benefit from decomposition.

These results suggest that the "price of universality" in lossless compression is not merely asymptotic overhead but can represent orders-of-magnitude performance gaps on highly regular structured binary data — gaps that simple, schema-free preprocessing can largely close. The schema-free nature of the width detection algorithm is its key practical advantage: unlike HDF5, Blosc, and Parquet, it requires no user configuration or data format knowledge, making it applicable as a transparent preprocessing layer in any compression pipeline.

Future work includes validation on binary datasets with estimated gaps in the 2×–10× range (USGS LiDAR point clouds [18], enterprise packet captures [19]) and formal bounds on the universality gap using LZ optimality theory [9].

Acknowledgments

This work was developed with substantial assistance from AI tools. The author conceived the byte-plane decomposition approach, the auto-detection algorithm, and the universality gap framework. Claude (Anthropic) was used extensively for literature research, theoretical analysis, experimental design feedback, and iterative manuscript

review. All experimental results were produced by the author's implementation and independently verified via automated reproducibility scripts.

References

[1] Y. Collet, "Zstandard compression," RFC 8478, 2018.

[2] J. Ziv and A. Lempel, "A universal algorithm for sequential data compression," IEEE Trans. Information Theory, vol. 23, no. 3, pp. 337–343, 1977.

[3] The HDF Group, "HDF5 shuffle filter," https://docs.hdfgroup.org/hdf5/.

[4] K. Masui et al., "Efficient compression of radio interferometric data," Astronomy and Computing, vol. 12, pp. 181–190, 2015.

[5] F. Alted, "Bytedelta: enhance your compression toolset," Blosc Blog, 2023, https://blosc.org/posts/bytedelta-enhance-compression-toolset/.

[6] Apache Software Foundation, "Apache Parquet format specification: BYTE_STREAM_SPLIT encoding," https://parquet.apache.org/.

[7] TDT: Typed Data Transformation, arXiv:2506.18062, 2025.

[8] M. Hershcovitch et al., "ZipNN: lossless compression for AI models," arXiv:2411.05239, 2024.

[9] P. Ferragina, I. Nitto, and R. Venturini, "On the bit-complexity of Lempel-Ziv compression," SIAM J. Computing, vol. 42, no. 4, pp. 1521–1541, 2013.

[10] Silesia Compression Corpus, https://sun.aei.polsl.pl/~sdeor/index.php?page=silesia.

[11] A. Painsky, S. Rosset, and M. Feder, "Generalized independent component analysis over finite alphabets," IEEE Trans. Information Theory, vol. 62, no. 2, pp. 1038–1053, 2016.

[12] P. Grünwald, The Minimum Description Length Principle, MIT Press, 2007.

[13] N. Vereshchagin and P. Vitányi, "Kolmogorov's structure functions and model selection," IEEE Trans. Information Theory, vol. 50, no. 12, pp. 3265–3290, 2004.

[14] D. Abadi, S. Madden, and M. Ferreira, "Integrating compression and execution in column-oriented database systems," Proc. SIGMOD, pp. 671–682, 2006.

[15] D. Abadi, P. Boncz, and S. Harizopoulos, "Column-oriented database systems," Proc. VLDB Endowment, vol. 2, no. 2, pp. 1664–1665, 2009.

[16] P. Bodik et al., "Intel Berkeley Research Lab sensor data," 2004, http://db.csail.mit.edu/labdata/labdata.html.

[17] NASDAQ, "ITCH 5.0 total view data," https://emi.nasdaq.com/ITCH/Nasdaq%20ITCH/.

[18] USGS, "3D Elevation Program (3DEP) LiDAR data," https://apps.nationalmap.gov/downloader/.

[19] R. Pang et al., “The devil and packet trace anonymization,” ACM Computer Communication Review, vol. 36, no. 1, pp. 29–38, 2006.